

0.58.3. La fonction spirale() — tourner en spirale

#include <spirale> : spirale(x, y, tuile, taille, sens, vitesse) part du centre et s'en éloigne en tournant.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `spirale(...)` part du centre et **tourne en s'éloignant** — des branches de 1, 1, 2, 2, 3, 3... pas — jusqu'au bord d'un carré de `taille`, puis revient au centre.

La ligne à écrire : `#include <spirale>`.

Ses arguments : le centre, la tuile, la taille, le sens, la vitesse.

Ce qu'elle coûte : environ **500 octets**, plus la police entière.

Essaie : le sens -1.

On retrouve spirale dans 8 leçons

[0.59](#), [0.59.1](#), [0.59.2](#), [0.63](#), [0.63.1](#), [0.63.2](#), [0.64](#), [0.64.2](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1347 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
spirale()	442 o	← main() — une fonction fournie par le langage
main()	34 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 3	39 o sur place, dans spirale() + AdresseCase (10 o)
effacer()	× 2	30 o sur place, dans spirale() + AdresseCase (10 o), EffacerCases (12 o)
image()	× 3	9 o sur place, dans spirale(), main() + AttendreImage (28 o)

```
Tes fonctions appelées :  
spirale() × 1 27 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

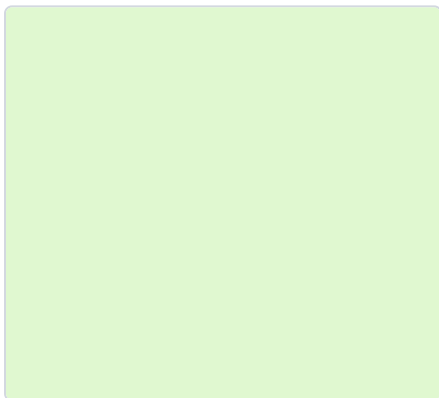
```
// ---- #include <spirale> : tourner en spirale ----  
  
#include <spirale> // spirale() : une spirale depuis le centre  
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

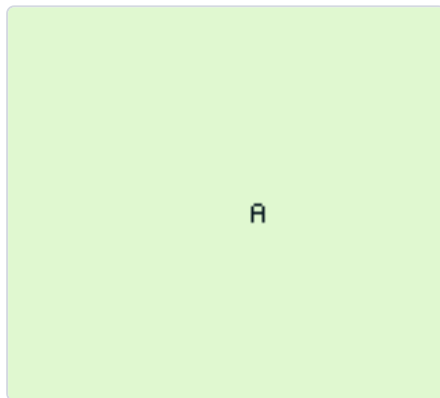
```
int main() {  
    spirale(10, 8, ALPHABET[0], 2, 1, 100);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

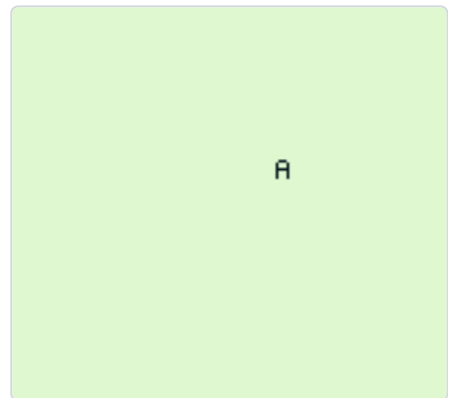
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui tourne en s'éloignant du milieu, puis y revient.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A s'éloigne jusqu'à 2 cases du centre

✓ il revient au centre (10,8)

Ensuite : **0.59. Tourner en s'éloignant : spirale** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++